



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس مبانی رایانش توزیع شده
تمرین اول (۱)

استاد درس

محمدرضا شکورنیا

دستیار ارشد

پویا جمشیدی

دستیاران مسئول تمرین

فرشته باقری

سبحان علاءالدینی

بهار ۱۴۰۵

فهرست مطالب

۳	۱ هدف تمرین
۵	۲ بخش اول: ارتباط بین دو پردازش مستقل با Pipe
۹	۳ بخش دوم: همزمانی، زمانبندی و مشاهده اثرات Switching Context
۱۳	۴ بخش سوم: ساخت سرویس، کانتینری سازی و اجرا در ماشین مجازی
۱۶	۵ بخش گزارش
۱۶	۶ معیار ارزیابی
۱۸	۷ تمایز گروه ها
۱۸	۸ دستاوردهای تمرین

۱ هدف تمرین

هدف این تمرین آشنایی عملی با سه مفهوم مهم در سیستم های توزیع شده و سیستم عامل است:

- ارتباط بین پردازش ها *Inter-Process Communication*
- همزمانی، زمانبندی و مشاهده اثرات *Scheduling* در برنامه های چند اجرایی
- استقرار یک سرویس ساده در محیط کانتینری و اجرای آن در ماشین مجازی

این تمرین عمداً به شکلی طراحی شده است که هر سه بخش آن به هم مرتبط باشند. شما در بخش اول یک سرویس محاسباتی ساده را با *Pipe* بین دو پردازش پیاده سازی می کنید، در بخش دوم رفتار همزمانی و اثرات زمانبندی را تحلیل می کنید، و در بخش سوم یک نسخه شبکه ای از همان ایده را در قالب یک سرویس *HTTP* کانتینری شده داخل ماشین مجازی اجرا می کنید.

نکته مهم: در این تمرین، هدف فقط تولید خروجی درست نیست. بخش مهمی از نمره به طراحی مناسب، کیفیت پیاده سازی، مدیریت خطا، توانایی تحلیل و قابل بازتولید بودن نتایج اختصاص دارد.

۱-۱ قوانین کلی

- زبان برنامه نویسی تمامی بخش ها فقط *Go* است.
- محیط توسعه و اجرا باید لینوکس باشد.
- استفاده از کتابخانه های خارج از *Standard Library* مجاز نیست مگر با تایید صریح تیم درس.
- برای دریافت ماژول ها و پکیج های *Go* باید *GOPROXY* را مطابق دستور زیر تنظیم کنید:

```
go env -w GOPROXY=https://mirror-go.runflare.com
```

- برای بررسی صحت تنظیم:

```
go env GOPROXY
```

- در صورتی که به هر دلیل نیاز به *fallback* داشتید، می توانید از فرم زیر استفاده کنید:

```
go env -w GOPROXY=https://mirror-go.runflare.com|direct
```

- با این حال، در این تمرین تا جایی که ممکن است باید از همان *mirror* تعیین شده استفاده کنید.
- کد ارسالی باید به صورت کامل قابل اجرا باشد و به هیچ فایل یا تنظیم دستی مبهم وابسته نباشد.
- هر بخش باید شامل *README* با دستورات اجرا، وابستگی ها، و توضیح ساختار فایل ها باشد.

۲-۱ ساختار تحویل

فایل های ارسالی باید به صورت زیر سازماندهی شوند:

HW1/

part1/

interface.go

worker.go

README.md

part2/

main.go

README.md

results/

part3/

main.go

Dockerfile

README.md

report.pdf

گزارش باید شامل توضیح مختصر طراحی، دستورات اجرا، نتایج، جدول ها، نمودارها، و تحلیل شما باشد.

۲ بخش اول: ارتباط بین دو پردازش مستقل با Pipe

۱-۲ شرح مسئله

در این بخش باید دو *process* مستقل در *Go* پیاده سازی کنید که با استفاده از *Pipe* با یکدیگر در ارتباط باشند:

- پردازش اول: *Interface Process*

- پردازش دوم: *Worker Process*

پردازش *Interface* نقش نقطه ورود درخواست را دارد. این پردازش درخواست را از کاربر دریافت می کند، آن را از طریق *Pipe* برای *Worker* ارسال می کند، و پس از دریافت پاسخ نتیجه را نمایش می دهد. پردازش *Worker* باید به صورت مداوم در حال اجرا و آماده دریافت درخواست باشد. این پردازش پس از دریافت هر درخواست، آن را پردازش کرده و نتیجه را باز می گرداند.

۲-۲ هدف آموزشی

- درک تفاوت بین *thread* و *process*
- آشنایی با *IPC* و ارتباط بین پردازش های مستقل
- درک مفهوم *request/response* در یک سرویس ساده
- تمرین طراحی پروتکل ساده و مقاوم در برابر خطا

۳-۲ نیازمندی های اجباری

۱. باید دو برنامه مستقل داشته باشید، نه یک برنامه که دو تابع را پشت سر هم اجرا کند.

۲. ارتباط بین این دو باید با مکانیسم *Pipe* انجام شود.

۳. *Worker* باید پس از اجرا در حالت گوش دادن باقی بماند و بتواند چندین درخواست متوالی را بدون راه اندازی مجدد پردازش کند.

۴. *Interface* باید از کاربر ورودی بگیرد، آن را به *Worker* ارسال کند و پاسخ را نمایش دهد.

۵. *Worker* باید بتواند حداقل چهار عمل زیر را پشتیبانی کند:

• *ADD*

• *SUB*

• *MUL*

• *DIV*

۶. هر درخواست باید مستقل از درخواست قبلی پردازش شود.

۷. در صورت بروز خطا، برنامه نباید بدون پیام مناسب متوقف شود.

۴-۲ فرمت درخواست

برای سادگی و قابل ارزیابی بودن، فرمت درخواست را به صورت زیر تعریف کنید:

OP A B

که در آن:

• *OP* نام عمل است

• *A* و *B* دو عملوند عددی هستند

نمونه های معتبر:

ADD 5 7

SUB 10 3

MUL 4 6

DIV 8 2

۵-۲ فرمت پاسخ

پاسخ *Worker* باید به صورت یک رشته متنی استاندارد یا *JSON* ساده به *Interface* برگردانده شود. استفاده از *JSON* امتیاز طراحی بهتری دارد، اما اجباری نیست. نمونه پاسخ متنی:

OK 12

ERR division_by_zero

۶-۲ مدیریت خطاهای اجباری

حداقل خطاهای زیر باید مدیریت شوند:

- دستور ناشناخته
- تعداد آرگومان نادرست
- ورودی غیر عددی
- تقسیم بر صفر
- قطع شدن *Pipe*
- اجرا نبودن *Worker*
- بسته شدن غیر منتظره پردازش مقابل

موارد غیرمجاز

- استفاده از *socket* به جای *Pipe* در این بخش
- استفاده از یک فایل معمولی به عنوان جایگزین *IPC*
- ترکیب هر دو نقش *Interface* و *Worker* در یک پردازش

۷-۲ خروجی مورد انتظار

در *README* این بخش باید موارد زیر مشخص باشد:

- چگونه *Worker* را اجرا می کنید
- چگونه *Interface* را اجرا می کنید
- ترتیب صحیح اجرا
- چند نمونه ورودی و خروجی
- توضیح مختصر پروتکل تبادل پیام

۸-۲ پیشنهاد فنی

برای این بخش استفاده از *named pipe* در لینوکس پیشنهاد می شود. همچنین استفاده از پکیج های استاندارد زیر مفید خواهد بود:

- *os*
- *bufio*
- *fmt*
- *strings*
- *strconv*
- *encoding/json*
- *log*

۳ بخش دوم: همزمانی، زمانبندی و مشاهده اثرات Switch- Contexting

۱-۳ شرح مسئله

در این بخش باید برنامه ای بنویسید که تعداد زیادی واحد اجرایی همزمان را در Go اجرا کند و اثرات افزایش همزمانی را بر کارایی سیستم مشاهده و تحلیل نماید. نکته بسیار مهم: در Go شما قرار نیست *OS-level context switch* را به صورت مستقیم پیاده سازی کنید. هدف این بخش مشاهده و تحلیل اثرات زمانبندی، رقابت برای CPU، افزایش تعداد *goroutine* ها، و هزینه های ناشی از *switching* و *synchronization* است.

۲-۳ هدف آموزشی

- درک تفاوت *thread* و *goroutine*
- آشنایی با مفهوم *scheduler* در Go
- مشاهده این که افزایش همزمانی همیشه به معنای افزایش کارایی نیست
- تمرین طراحی آزمایش و استخراج متریک های عملکردی

۳-۳ نیازمندی های اجباری

برنامه شما باید حداقل سه پارامتر زیر را تغییر دهد و اثر آن ها را بر عملکرد گزارش کند:

۱. تعداد *goroutine* ها

۲. نوع *workload*

۳. مقدار *GOMAXPROCS*

۴-۳ مقادیر اجباری برای آزمایش

حداقل آزمایش ها باید برای تعداد *goroutine* های زیر انجام شوند:

1, 2, 4, 8, 16, 32, 64

همچنین باید حداقل سه تنظیم مختلف برای *GOMAXPROCS* بررسی شود:

1 •

2 •

runtime.NumCPU() •

۵-۳ نوع *workload*

شما باید حداقل دو سناریوی متفاوت را پیاده سازی و مقایسه کنید:

سناریوی اول: **workload محاسباتی** هر *goroutine* یک کار صرفا محاسباتی انجام دهد. برای مثال:

• تکرار یک حلقه سنگین

• محاسبه مجموع تعداد زیادی عدد

• انجام یک محاسبه عددی تکراری

سناریوی دوم: **workload ترکیبی** هر *goroutine* علاوه بر محاسبه، شامل یک مرحله انتظار یا رقابت بر سر منبع مشترک باشد. برای مثال:

• محاسبه کوتاه + *sleep*

• محاسبه + ارسال روی *channel*

• محاسبه + رقابت بر سر *mutex*

۳-۶ متریک های اجباری

حداقل متریک های زیر باید برای هر تنظیم گزارش شوند:

- زمان کل اجرا
- *throughput*
- تاخیر متوسط هر کار
- تعداد *goroutine*
- مقدار *GOMAXPROCS*

در صورتی که خواستید می توانید متریک های زیر را نیز اضافه کنید:

- حداکثر تاخیر
- حداقل تاخیر
- انحراف معیار زمان اجرا
- تعداد دفعات بلاک شدن

۳-۷ تحلیل مورد انتظار

در گزارش خود باید حداقل به سوالات زیر پاسخ دهید:

۱. با افزایش تعداد *goroutine* ها چه اتفاقی برای زمان اجرا می افتد؟
۲. آیا افزایش تعداد *goroutine* ها همیشه باعث افزایش *throughput* می شود؟
۳. تفاوت رفتار برنامه در $GOMAXPROCS=1$ و $GOMAXPROCS=NumCPU$ چیست؟
۴. در کدام نوع *workload* اثرات زمانبندی و *switching* بیشتر دیده می شود؟
۵. از چه نقطه ای به بعد، افزایش همزمانی مفید نیست یا حتی کارایی را کاهش می دهد؟

۸-۳ استفاده از ابزارهای مشاهده

استفاده از ابزارهای استاندارد *Go* برای این بخش به شدت توصیه می شود. به ویژه:

• *runtime*

• *runtime/trace*

• *time*

• *sync*

در صورت استفاده از *trace*، باید در *README* توضیح دهید که چگونه فایل *trace* تولید شده و چگونه آن را مشاهده می کنید.

۹-۳ خروجی گزارش

برای این بخش وجود موارد زیر در گزارش اجباری است:

• حداقل یک جدول از نتایج

• حداقل یک نمودار

• تحلیل متنی نتایج

• توضیح این که چرا برخی تنظیم ها بهتر یا بدتر عمل کرده اند

نمونه ساختار جدول:

<i>Throughput</i>	<i>Total Time</i>	<i>Workload</i>	<i>GOMAXPROCS</i>	<i>Goroutines</i>
...	...	<i>CPU-bound</i>	<i>1</i>	<i>1</i>
...	...	<i>Mixed</i>	<i>2</i>	<i>8</i>

۱۰-۳ موارد غیرمجاز

- صرفاً چاپ چند عدد بدون هیچ تحلیل
- اجرای برنامه فقط برای یک تعداد ثابت *goroutine*
- حذف مقایسه بین چند مقدار مختلف *GOMAXPROCS*
- ارائه نمودار بدون تفسیر

۴ بخش سوم: ساخت سرویس، کانتینری سازی و اجرا در ماشین مجازی

۱-۴ شرح مسئله

در این بخش باید یک سرویس ساده *HTTP* در *Go* پیاده سازی کنید، آن را در قالب یک *Docker image* بسازید، سپس این کانتینر را در داخل یک ماشین مجازی اجرا کنید و از سیستم *Host* به آن متصل شوید.

۲-۴ هدف آموزشی

- آشنایی با تبدیل یک برنامه به سرویس شبکه ای
- آشنایی با *containerization*
- آشنایی با اجرای سرویس در یک محیط ایزوله
- درک ارتباط بین *guest VM* و *host*

۳-۴ نیازمندی های اجباری سرویس

سرویس شما باید حداقل دو *endpoint* زیر را داشته باشد:

- *GET /health*
- *GET /compute*

`/health` این *endpoint* برای بررسی سلامت سرویس است و باید پاسخ ساده و معتبر برگرداند.

`/compute` این *endpoint* باید یک درخواست محاسباتی را از طریق *query parameter* دریافت کرده و پاسخ را به صورت *JSON* برگرداند.
نمونه درخواست:

`/compute?op=add&a=5&b=7`

نمونه پاسخ:

```
{  
  "operation": "add",  
  "a": 5,  
  "b": 7,  
  "result": 12  
}
```

۴-۴ مدیریت خطاهای اجباری

حداقل خطاهای زیر باید مدیریت شوند:

- نبودن پارامترها
- نامعتبر بودن *op*
- عددی نبودن *a* یا *b*
- تقسیم بر صفر
- استفاده از متد نادرست

۵-۴ نیازمندی های اجباری کانتینری سازی

۱. باید *Dockerfile* بنویسید.
۲. باید بتوان با یک دستور *image* را ساخت.
۳. باید بتوان کانتینر را داخل ماشین مجازی اجرا کرد.
۴. پورت سرویس باید به شکلی تنظیم شود که از *Host* قابل دسترسی باشد.
۵. باید اثبات کنید که درخواست از *Host* به سرویس داخل *VM* ارسال شده و پاسخ دریافت شده است.

۶-۴ موارد اجباری در README این بخش

- دستور ساخت *Docker image*
- دستور اجرای کانتینر
- شماره پورت استفاده شده
- روش اتصال از *Host* به *VM*
- نمونه دستور *curl* یا نمونه فراخوانی از مرورگر

۷-۴ اثبات اجرا

- برای این بخش باید حداقل مدارک زیر را ارائه کنید:
- تصویر یا خروجی ترمینال از اجرای کانتینر داخل *VM*
 - تصویر یا خروجی ترمینال از فراخوانی موفق */health*
 - تصویر یا خروجی ترمینال از فراخوانی موفق */compute*

۸-۴ پیشنهاد فنی

استفاده از *net/http* و *encoding/json* برای این بخش کافی است و نیازی به فریمورک های خارجی ندارید.

۵ بخش گزارش

علاوه بر کد، هر گروه باید یک گزارش مختصر و فنی تحویل دهد. این گزارش باید به صورت *PDF* باشد و حداقل شامل بخش های زیر باشد:

- توضیح معماری و طراحی هر سه بخش
- توضیح پروتکل ارتباطی در بخش اول
- توضیح سناریوهای آزمایش در بخش دوم
- جدول ها و نمودارهای بخش دوم
- تحلیل نتایج
- توضیح مراحل ساخت و اجرای بخش سوم
- چالش ها و مشکلاتی که در حین پیاده سازی داشته اید

۶ معیار ارزیابی

بخش اول: 35 نمره

- طراحی صحیح دو پردازش مستقل: 8
- پیاده سازی صحیح ارتباط با *Pipe*: 10
- پایداری *Worker* و پشتیبانی از چند درخواست متوالی: 6

- مدیریت خطا: 6

- ساختار کد و مستندسازی: 5

بخش دوم: 35 نمره

- پیاده سازی صحیح برنامه همزمان: 10

- طراحی مناسب آزمایش: 8

- استخراج و گزارش متریک ها: 7

- جدول و نمودار: 5

- تحلیل فنی و استدلال: 5

بخش سوم: 35 نمره

- پیاده سازی صحیح سرویس *HTTP*: 10

- *Dockerfile* و کانتینری سازی صحیح: 10

- اجرا در ماشین مجازی: 9

- اتصال موفق از *Host* به *Guest*: 6

کسری نمره

موارد زیر باعث کسری نمره خواهند شد:

- نبودن *README* یا ناقص بودن آن

- نبودن دستور اجرای دقیق

- وابستگی به کتابخانه خارجی بدون اعلام

- خروجی درست بدون تحلیل در بخش دوم
- اجرا نشدن پروژه به دلیل تنظیمات مبهم
- نداشتن مدرک اجرای بخش سوم

۷ تمایز گروه ها

برای کاهش شباهت بین پاسخ ها، هر گروه باید حداقل یک قابلیت اضافه به انتخاب خود پیاده سازی کند. نمونه های مناسب:

- پشتیبانی از عمل MOD
- پشتیبانی از عمل POW
- پشتیبانی از عمل MAX
- پشتیبانی از عمل MIN
- استفاده از پاسخ $JSON$ در بخش اول
- ثبت log مناسب برای درخواست ها

۸ دستاوردهای تمرین

این تمرین باید نشان دهد که شما:

- تفاوت بین پردازش و واحدهای اجرای همزمان را می فهمید
- می توانید یک ارتباط IPC ساده اما صحیح طراحی کنید
- می توانید اثرات همزمانی و زمانبندی را با متریک و تحلیل بررسی کنید

● می توانید یک سرویس ساده را کانتینری کرده و در محیط مجازی اجرا نمایید

توجه: کیفیت تحلیل، دقت در مدیریت خطا، و قابل بازتولید بودن نتایج، بخش مهمی از ارزیابی خواهد بود.
در نتیجه فقط به تولید خروجی بسنده نکنید و به طراحی و مستندسازی نیز با دقت توجه کنید.